Log for Support Vector Machine

Student id: C00313459

Name: Amisha Das

1. Introduction

Support Vector Machine (SVM) notebook to work with 3 types of SVMs: Linear SVM (without Kernel), SVM with Kernel and a kernel SVM with PCA

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

In [3]:
```
df = pd.read_csv('sdss_star_classification.csv')
```

About data-

Sloan Digital Sky Survey (SDSS) is a sky survey that collects magnitude values in 5 bands (*u,g,r,i* and *z*) and redshift values for stars, quasars and galaxies.

In [4]:
```
display(df.head())
```

	obj_ID	alpha	delta	u	g	r	i	z	run_ID	rerun_ID	cam_col	field_ID	spec_obj_ID	class	redshift
0	1.237661e+18	135.689107	32.494632	23.87882	22.27530	20.39501	19.16573	18.79371	3606	301	2	79	6.543777e+18	GALAXY	0.634794
1	1.237665e+18	144.826101	31.274185	24.77759	22.83188	22.58444	21.16812	21.61427	4518	301	5	119	1.176014e+19	GALAXY	0.779136
2	1.237661e+18	142.188790	35.582444	25.26307	22.66389	20.60976	19.34857	18.94827	3606	301	2	120	5.152200e+18	GALAXY	0.644195
3	1.237663e+18	338.741038	-0.402828	22.13682	23.77656	21.61162	20.50454	19.25010	4192	301	3	214	1.030107e+19	GALAXY	0.932346
4	1.237680e+18	345.282593	21.183866	19.43718	17.58028	16.49747	15.97711	15.54461	8102	301	3	137	6.891865e+18	GALAXY	0.116123

In [4]:
```
df['class'] = df['class'].map({'STAR': 0, 'GALAXY': 1, 'QSO': 2})
```

Features and target variable

In [5]:
```
X = df.drop(columns=['class'])
y = df['class']
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

In [7]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. Linear SVM (without Kernel)

In [8]:
```
svm_linear = SVC(kernel='linear', C=1.0)
svm_linear.fit(X_train, y_train)
```

Out[8]:
```
SVC(kernel='linear')
```

Predictions:

In [9]:
```
y_pred_linear = svm_linear.predict(X_test)
```

In [10]:
```
accuracy_linear = accuracy_score(y_test, y_pred_linear)
print(f"Linear SVM Model Accuracy: {accuracy_linear:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred_linear))
```

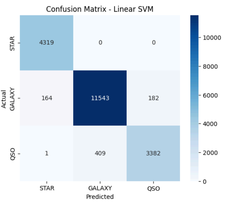
```
Linear SVM Model Accuracy: 0.9622
Classification Report:
              precision    recall  f1-score   support

           0       0.96      1.00      0.98      4319
           1       0.97      0.97      0.97     11889
           2       0.95      0.89      0.92      3792

    accuracy                           0.96     20000
   macro avg       0.96      0.95      0.96     20000
weighted avg       0.96      0.96      0.96     20000
```

Confusion Matrix for linear SVM

In [11]:
```
plt.figure(figsize=(6,5))
sns.heatmap(confusion_matrix(y_test, y_pred_linear), annot=True, fmt='d', cmap='Blues', xticklabels=['STAR', 'GALAXY', 'QSO'], yticklabels=['STAR', 'GALAXY', 'QSO'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Linear SVM")
plt.show()
```



Conclusion

Excellent performance with 96% accuracy and a strong diagonal in the confusion matrix

3. SVM with Kernel

In [12]:
```
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_model.fit(X_train, y_train)
```

Out[12]:
```
SVC()
```

In [13]:
```
y_pred = svm_model.predict(X_test)
```

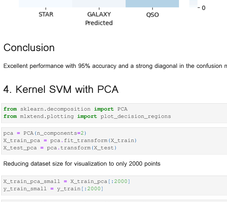
In [14]:
```
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Model Accuracy: 0.9580
Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.97      0.96      4319
           1       0.96      0.97      0.97     11889
           2       0.97      0.90      0.93      3792

    accuracy                           0.96     20000
   macro avg       0.96      0.95      0.95     20000
weighted avg       0.96      0.96      0.96     20000
```

In [15]:
```
plt.figure(figsize=(6,5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues', xticklabels=['STAR', 'GALAXY', 'QSO'], yticklabels=['STAR', 'GALAXY', 'QSO'])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Conclusion

Excellent performance with 95% accuracy and a strong diagonal in the confusion matrix however, performance is 1% worse than linear SVM.

4. Kernel SVM with PCA

In [17]:
```
from sklearn.decomposition import PCA
from mlxtend.plotting import plot_decision_regions
```

In [18]:
```
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

Reducing dataset size for visualization to only 2000 points

In [19]:
```
X_train_pca_small = X_train_pca[:2000]
y_train_small = y_train[:2000]
```

In [20]:
```
y_train_small_np = y_train_small.to_numpy()
```

Training with only stars and galaxies as previous reusits had a lot of problems with quasars

In [22]:
```
mask = (y_train_small_np == 0) | (y_train_small_np == 1)
X_train_pca_small_filtered = X_train_pca_small[mask]
y_train_small_filtered = y_train_small_np[mask]
```

Training a smaller SVM model

In [23]:
```
svm_pca_small = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_pca_small.fit(X_train_pca_small_filtered, y_train_small_filtered)
```

Out[23]:
```
SVC()
```

In [24]:
```
y_pred_pca = svm_pca_small.predict(X_train_pca_small_filtered)
```

In [25]:
```
print("Classification Report for PCA-Based SVM:")
print(classification_report(y_train_small_filtered, y_pred_pca, zero_division=0))
```

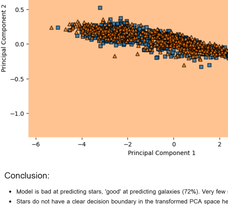
```
Classification Report for PCA-Based SVM:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00       447
           1       0.72      1.00      0.84      1167

    accuracy                           0.72      1614
   macro avg       0.36      0.50      0.42      1614
weighted avg       0.52      0.72      0.61      1614
```

Plotting decision boundary with fewer points

In [28]:
```
plt.figure(figsize=(8,6))
plot_decision_regions(X_train_pca_small_filtered, y_train_small_filtered, clf=svm_pca_small, legend=2)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("SVM Decision Boundary (PCA-Reduced Features for 2 Classes)")
plt.show()
```



Conclusion:

- Model is bad at predicting stars, 'good' at predicting galaxies (72%). Very few stars (feature 0) and SVM is biased to majority figure (feature 1, galaxies).
- Stars do not have a clear decision boundary in the transformed PCA space hence there are far more galaxy classifications.

Future work:

Increase accuracy of PCA SVM.

Log

In [71]:
```
log = pd.read_csv("log.csv")
pd.set_option('display.max_colwidth', None)
log
```

Out[71]:

	Description	Level of Difficulty	Change From	Changed To
0	Fixed missing module error (mlxtend)	Easy	ModuleNotFoundError	Installed mlxtend using !pip install mlxtend
1	Reduced PCA sample size to speed up visualization	Medium	pca = PCA(n_components=2)\nX_train_pca = pca.fit_transform(X_train)\nX_test_pca = pca.transform(X_test)	X_train_pca_small = X_train[:2000]\ny_train_small = y_train[:2000]
2	Fixed ValueError in plot_decision_regions	Medium	plot_decision_regions(X_train, y_train.to_numpy(), clf=svm_pca, legend=2)	Converted y_train to a NumPy array: y_train_small_np = y_train_small.to_numpy()
3	Fixed classification report error	Medium	print(classification_report(y_train_small_filtered, y_pred_pca))	print(classification_report(y_train_small_filtered, y_pred_pca, zero_division=0))